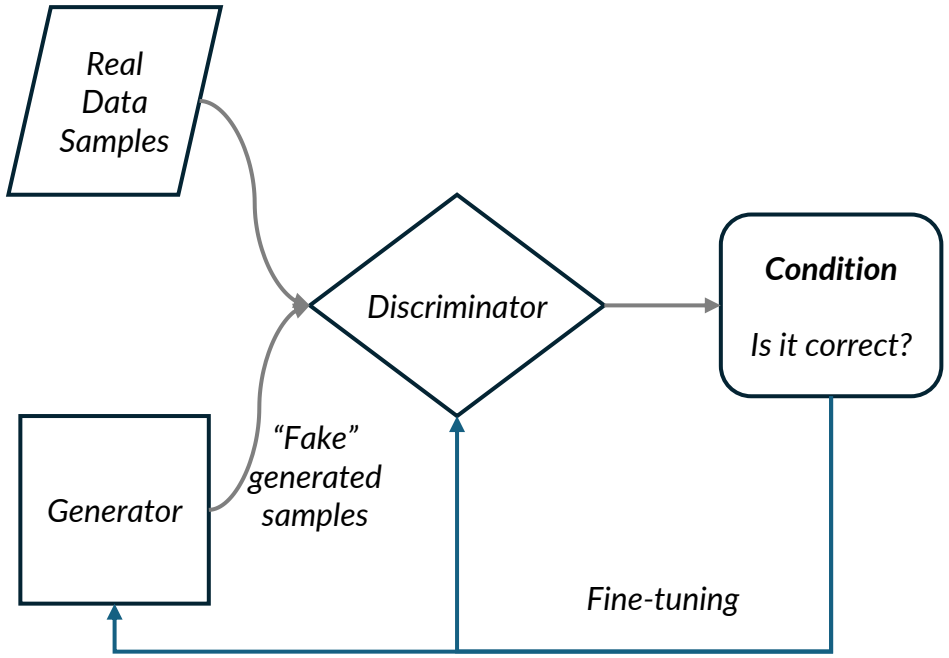
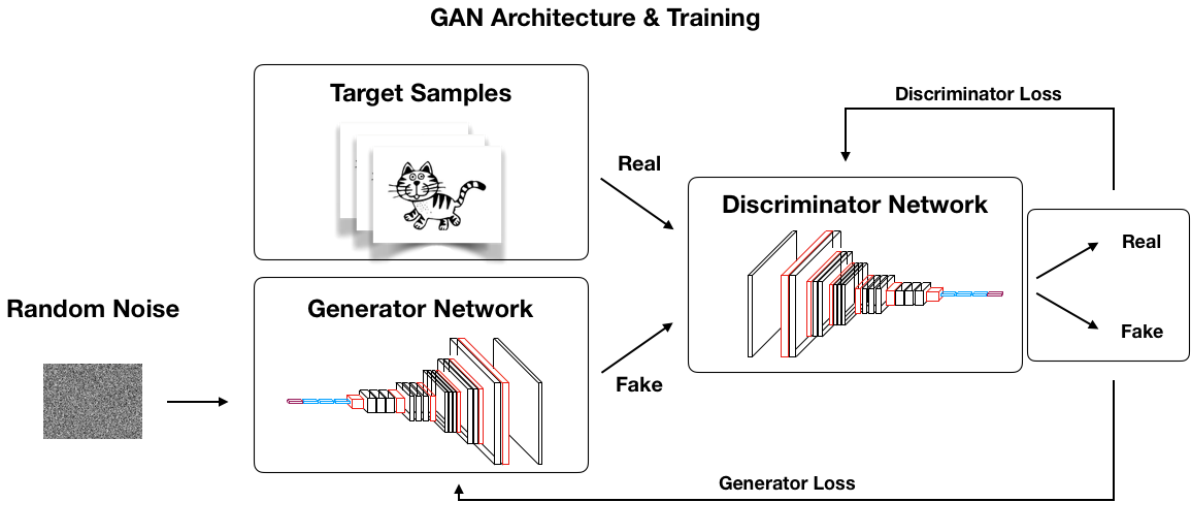
# MOVING BEYOND VAEs

- VAEs enable sampling, but likelihood-based training leads to averaging over multiple valid outputs, and produces smooth but unrealistic images
- Let's try changing our approach now: Can we generate realism without reconstruction?
- **Adversarial learning** uses a competitive process
- Two networks with opposing objectives
  - **Generator (*G*):** produces samples
  - **Discriminator (*D*):** judges realism
- There are no ground-truth labels for generated data
- Learning signal comes from being indistinguishable from real data



# GENERATIVE ADVERSARIAL NETWORK

- A Generative Adversarial Network (GAN) is an adversarial learning framework
- It has two neural networks competing against each other in a "zero-sum" game
  - the Generator, and
  - the Discriminator
- The goal of using a GAN is to produce synthetic data that is indistinguishable from real-world data



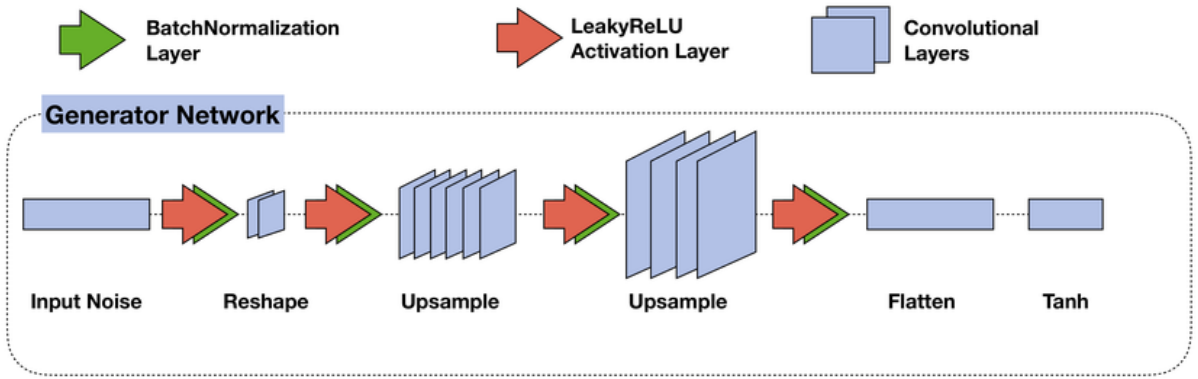

*Generator and Discriminator play a zero-sum game, one's gain is the other's loss*

# GENERATOR NETWORK

- The generator produces a fake image by taking a seed of random noise as input
- This random noise is a low-dimensional latent space vector of random numbers
- It transforms this random noise into an image in an iterative training process, improving the generation in each iteration
- Formally, the generator $G_\theta$ is a network that maps a latent random variable to data space:

$$\mathbf{z} \sim p(\mathbf{z}), \qquad \hat{\mathbf{x}} = G_\theta(\mathbf{z})$$

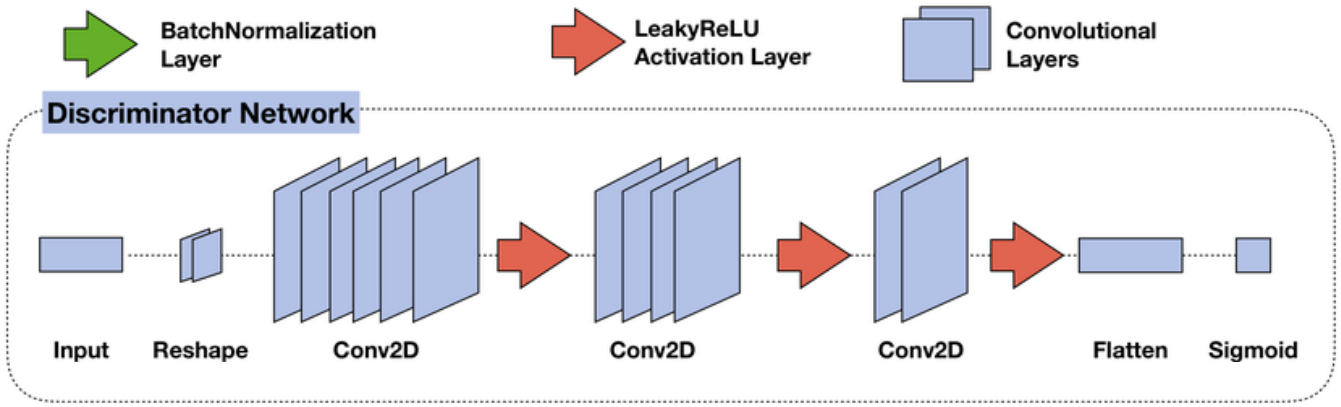
  - $\mathbf{z}$ is the latent noise vector
  - $p(\mathbf{z})$ is the simple, known prior
  - $\hat{\mathbf{x}}$ is the generated (fake) sample




*The generator learns a mapping from a low-dimensional latent space to a high-dimensional data manifold*

*It is a sampler - it learns the implicit structure and correlations in the data and a way to reparameterise distributions using noise*

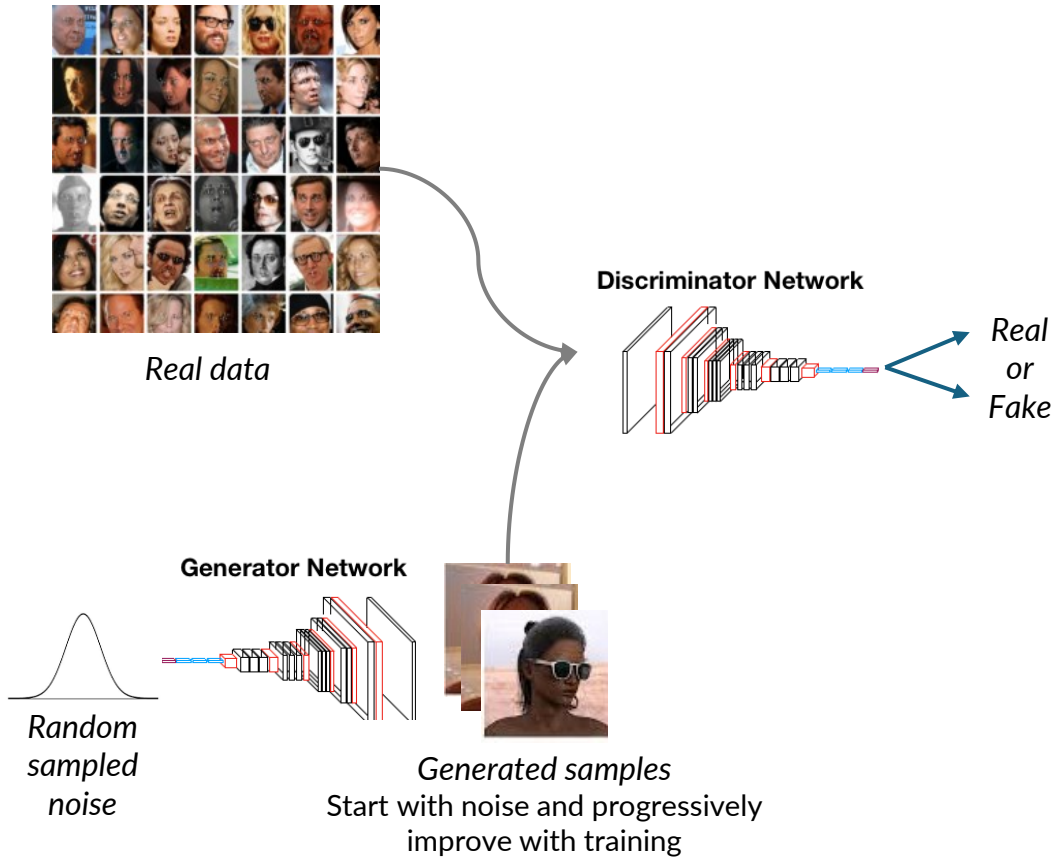
# DISCRIMINATOR NETWORK




- Discriminator is a binary classifier that outputs a probability that the input image is real
- The discriminator is a representation learner
- It is shown real images (from the training dataset) and fake images (from the generator)
- It compresses high-dimensional images into a single realism score
- This score defines what “realistic” means during training
- The generator doesn’t learn from real data, only from interaction with discriminator; However, the discriminator directly observes real data

# GAN WORKING - EXAMPLE

- Consider a target domain of human face images wearing sunglasses
- The generator takes a random noise vector sampled from a simple distribution and maps it to a synthetic face image
- The discriminator receives:
  - real images of people wearing sunglasses
  - generated images with synthetic sunglasses
- The discriminator learns to distinguish real from fake
- The generator updates its parameters to better fool the discriminator
- This competition continues until the generated images become indistinguishable from real ones

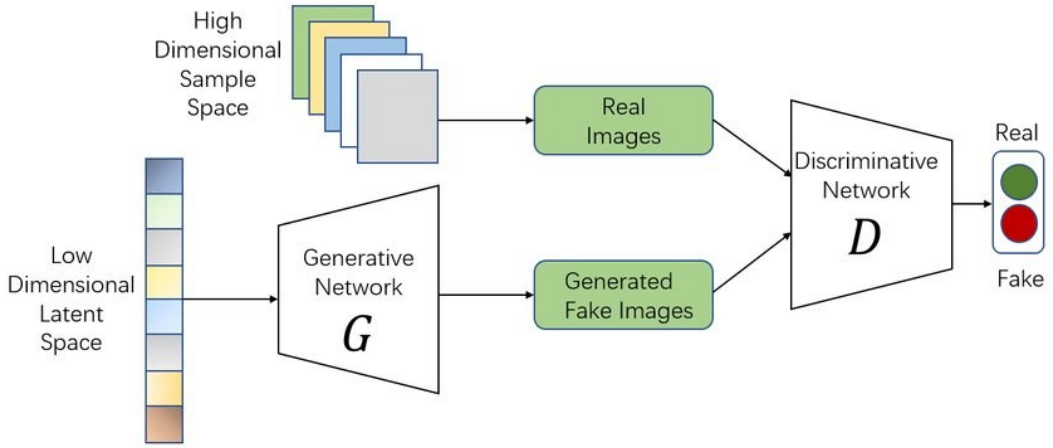


# USE CASES OF GANs

1. **Image generation and editing**
   - Generate realistic images from noise or text prompts
   - Modify images through super-resolution, colourisation, and style changes
   - Used in gaming, animation, virtual characters, and digital media
2. **Synthetic data generation**
   - Create realistic artificial data for training other ML models
   - Useful when real data is scarce, sensitive, or imbalanced
   - Examples include fraud detection, medical imaging, and anomaly simulation
3. **Data completion and inference**
   - Infer missing or unobserved information by learning correlations
   - Used in applications such as sub-surface imaging for energy and geospatial analysis
4. **3D generation from 2D inputs**
   - Generate 3D structures from 2D images or scans
   - Applied in healthcare for organ modelling, surgical planning, and simulation

# GAN - WORKING OVERVIEW

- The generator neural network analyses the training set and identifies data attributes
- The discriminator neural network also analyses the initial training data and distinguishes between the attributes independently
- The generator modifies some data attributes by adding noise (or random changes) to certain attributes
- The generator passes the modified data to the discriminator
- The discriminator calculates the probability that the generated output belongs to the original dataset
- If the discriminator catches the fake, the generator is penalised and adjusts its weights to do better next time
- If the discriminator is fooled, the discriminator is penalised and adjusts its weights to become more observant



# GAN OBJECTIVE

- GAN training is framed as a two-player minimax game between generator $G$ and discriminator $D$

$$\min_G \max_D \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p(z)}\left[\log\left(1 - D\big(G(z)\big)\right)\right]$$

- This calculates how well a generator can match the data distribution under an adversarial classifier, and uses that signal to iteratively improve both models
- More formally, the value of this objective measures how distinguishable the generated distribution $p_G$ is from the true data distribution $p_{\text{data}}$
- The generator's objective is to fool the discriminator, i.e., make $D\big(G(z)\big)$ close to 1; the loss measures this "foolability"
- Discriminator learns to improve the classification, while generator follows discriminator gradients

# ADVERSARIAL LEARNING SETUP

*The minimax indicates opposing objectives - the discriminator tries to maximise, while the generator tries to minimise the expression*

- The first expectation of the objective is the **real data term**

$$\mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)]$$

  - $x$ is a real data sample (image), $D(x) \in (0,1)$ is the discriminator's estimated probability that the image is real
  - This term rewards the discriminator for **assigning high probability to real data**

- The second expectation is the **generated** (fake) data term

$$\mathbb{E}_{z \sim p(z)}\left[\log\left(1 - D\big(G(z)\big)\right)\right]$$

  - $z$ is random noise sampled from a simple prior; $G(z)$ is the generated sample
  - $D\big(G(z)\big)$ is the probability that the discriminator assigns to the fake being real
  - $1 - D\big(G(z)\big)$ is large when $D$ confidently labels sample as fake
  - This term rewards the discriminator for correctly **rejecting fake samples**

# HOW DISCRIMINATOR LEARNS

- The discriminator is a parametric function

$$D_\phi : \mathbf{x} \rightarrow [0, 1]$$

- It learns via standard supervised learning, using labelled examples sampled from real data
- The objective for discriminator becomes

$$\max_D \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} \left[ \log \left( 1 - D\left(G(z)\right) \right) \right]$$

- Simply, the discriminator is actually maximising, $\mathbb{E}[\log D(\text{real})] + \mathbb{E}\left[\log\left(1 - D(\text{fake})\right)\right]$
- This is exactly the log-likelihood of a binary classifier, i.e., the standard binary cross-entropy objective, which maximises the probability of
  - Assigning high scores to real samples
  - Assigning low scores to generated samples

# HOW GENERATOR LEARNS

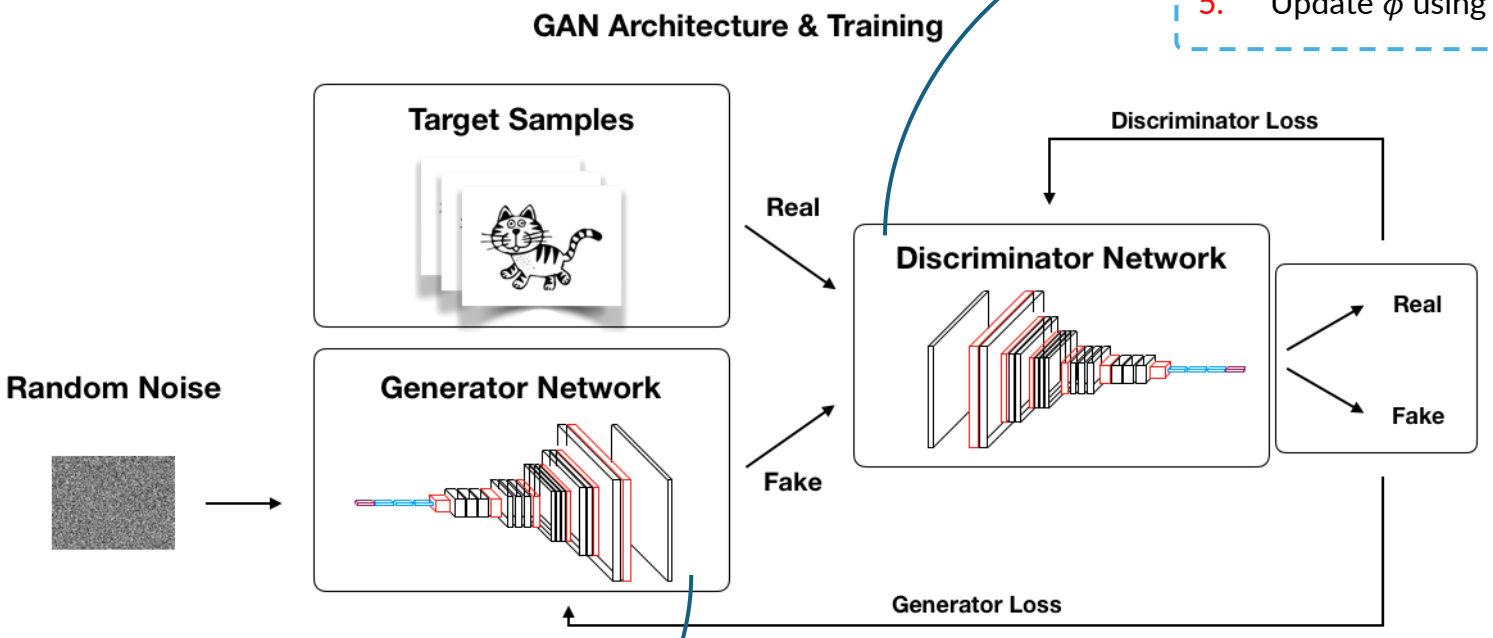
- The generator is a parametric function, which maps samples from a simple noise distribution $p(z)$ (e.g. Gaussian or uniform) to the data space
- The generator does not receive labelled data; It learns indirectly through feedback provided by the discriminator
- The generator affects only the second term

$$\min_G \mathbb{E}_{z\sim p(z)}\left[\log\left(1 - D\left(G(z)\right)\right)\right]$$

- Minimising this forces $D\left(G(z)\right)$ to be large, meaning the discriminator believes generated samples are real
- So, the generator learns to produce samples indistinguishable from real data under the discriminator
- The discriminator acts as a learned, adaptive loss function for the generator – its gradients push $G$ to transform regions of noise space into outputs that look more like real data

# COMBINED LEARNING FLOW

1. Sample a minibatch of real data $\{\mathbf{x}^{(i)}\}$
2. Take generate fake samples from noise $\{\boldsymbol{z}^{(i)}\}$
3. Compute discriminator predictions for both sets
4. Compute classification loss
5. Update $\phi$ using gradient descent




Trained by backpropagating through the discriminator:

1. Sample noise $z$
2. Generate fake data $\hat{\mathbf{x}} = \mathrm{G}_\theta(\mathbf{z})$
3. Pass $\hat{\mathbf{x}}$ to the discriminator
4. Compute generator loss based on discriminator output
5. Update $\theta$ via gradient descent

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}\left[\log\left(1 - D\left(G(z)\right)\right)\right]$$

# HOW TRAINING PROCEEDS

- Early in training, the discriminator easily separates real and fake data
- As the generator improves, fake samples move closer to the real data manifold
  - The discriminator's decision boundary becomes more refined
  - Discriminator gradients indicate how fake samples differ from real ones
  - These gradients are later reused (via backpropagation) to train the generator
- Training alternates between updating the discriminator and updating the generator
- Both objectives change continuously and there is no fixed optimum during training
- When the training converges
  - Discriminator can no longer distinguish real from fake
  - Generator distribution matches data distribution from the discriminator's view
- Over time, generated distribution $p_G$ moves closer to real data distribution $p_{\text{data}}$ and reaches Nash equilibrium

$$p_G = p_{\text{data}}, \qquad D(\mathbf{x}) = \frac{1}{2}$$

# WHY GANs PRODUCE SHARP IMAGES

- GANs do not minimise reconstruction error or likelihood
- There is no penalty for deviating from a specific training example
- The discriminator evaluates realism at the distribution level, not pixel level
- High-frequency details (edges, textures, local structure) strongly affect discriminator decisions
- As a result, the generator is pushed to reproduce fine visual details, not averages
- Sharpness in GANs is a direct consequence of adversarial realism, not architectural depth



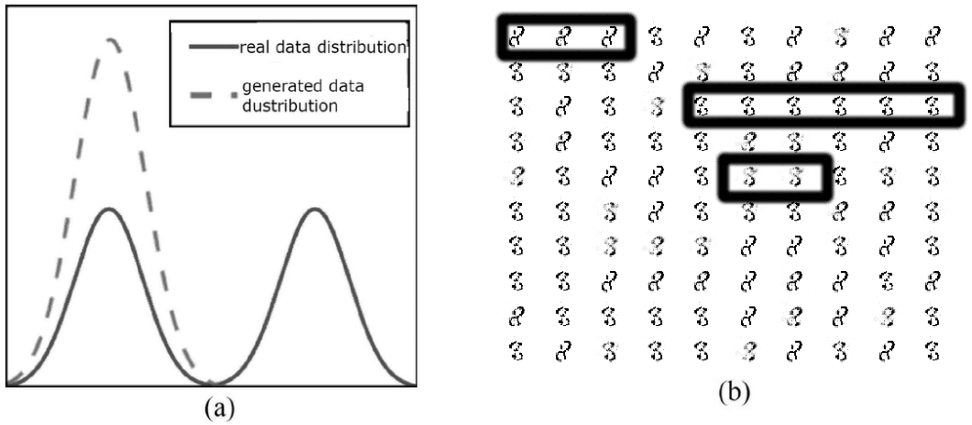
**VAE** **GAN**

Comparison between images generated by a VAE (left) and a GAN (right). VAEs tend to produce smoother but blurrier images due to their probabilistic decoding.

# FAILURE MODES – MODE COLLAPSE

- A generator does not inherently know how to create images, it samples random noise and learns how to map it to realistic images from the discriminator's set
- Producing diverse outputs is not explicitly required by the objective; it is rewarded whenever the image looks realistic
- If a small set of samples (mode) reliably fools the discriminator, the generator over-optimises for that
- Mode collapse occurs when the generator produces very similar outputs regardless of different input noise
- This leads to mode collapse: low diversity, high realism
- For example, if a dataset contains images of cats, dogs and birds, the generator might keep generating similar images of cats if they fool the discriminator easily

*A **mode** is a high-probability region in the data distribution*
*A good generator should cover **all modes***




*Instead of learning the entire data distribution, the generator can focus on a few modes*

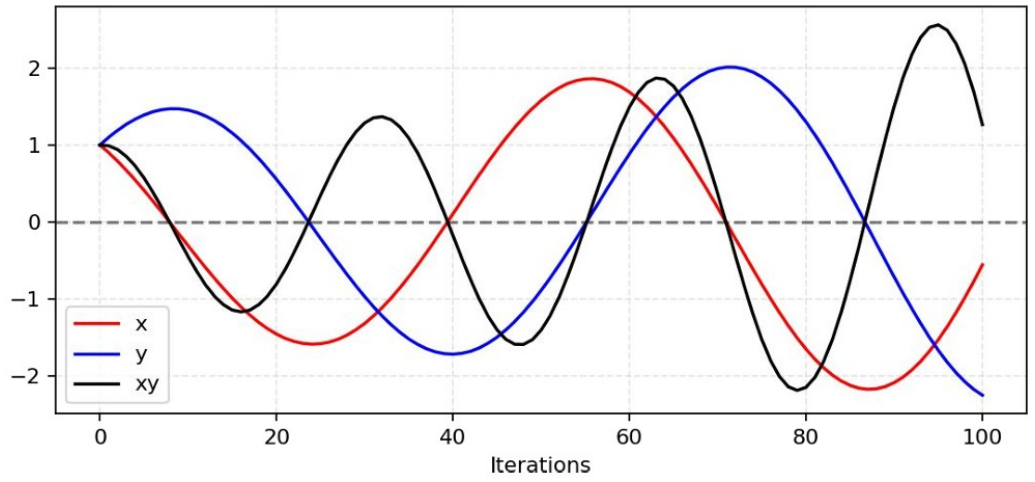
*This is why noise is essential – it introduces variability*

# FAILURE MODES – TRAINING INSTABILITY

- GAN training is a non-stationary minimax problem
- We saw during training that the objectives change continuously
- The model converges when the discriminator and the generator reach a Nash equilibrium
- If the discriminator becomes too strong, gradients vanish and generator stops learning
- If the generator improves too fast, discriminator becomes unreliable
- This can lead to divergence or cause the training to fail to converge

Two players ***D*** and ***G*** control the value of ***x*** and ***y*** respectively

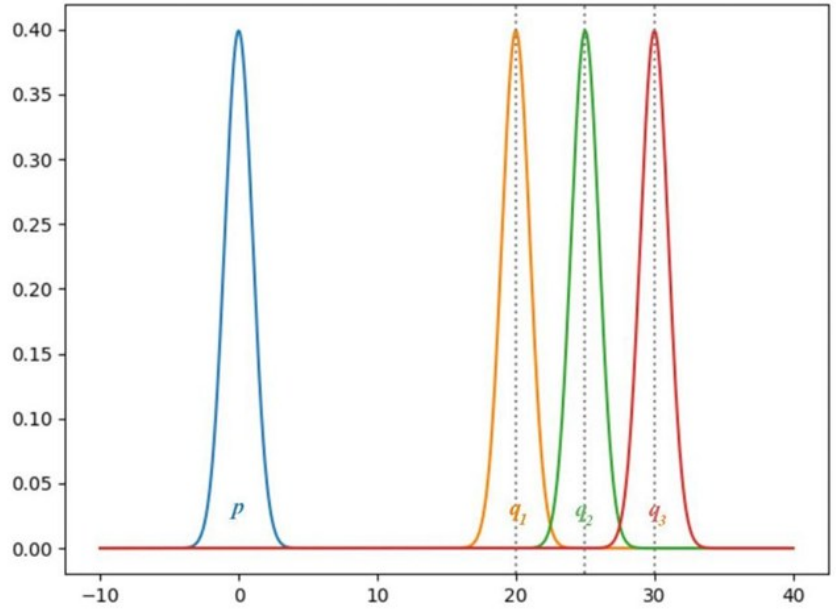
Player ***D*** wants to maximise the value ***xy*** while ***G*** wants to minimise it

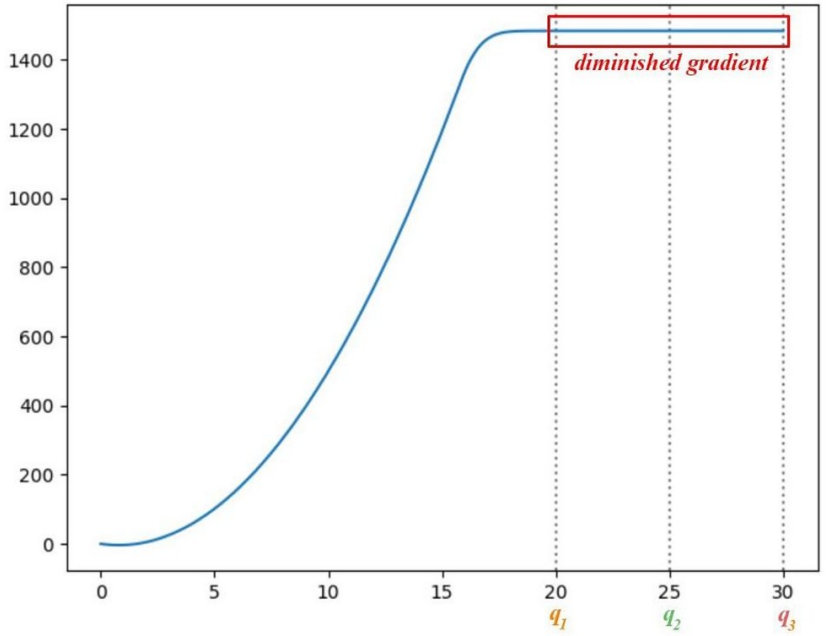



At equilibrium, $x=y=0$, the action of any player does not matter

# FAILURE MODES – VANISHING GRADIENTS

- If the discriminator is too good, then the generator training can fail due to vanishing gradients
- An optimal discriminator doesn't provide enough information for the generator to make progress
- The image shows how gradients will vanish if the distribution of generated images $(p)$ is too different than the distribution of real images $(q_1, q_2, q_3)$



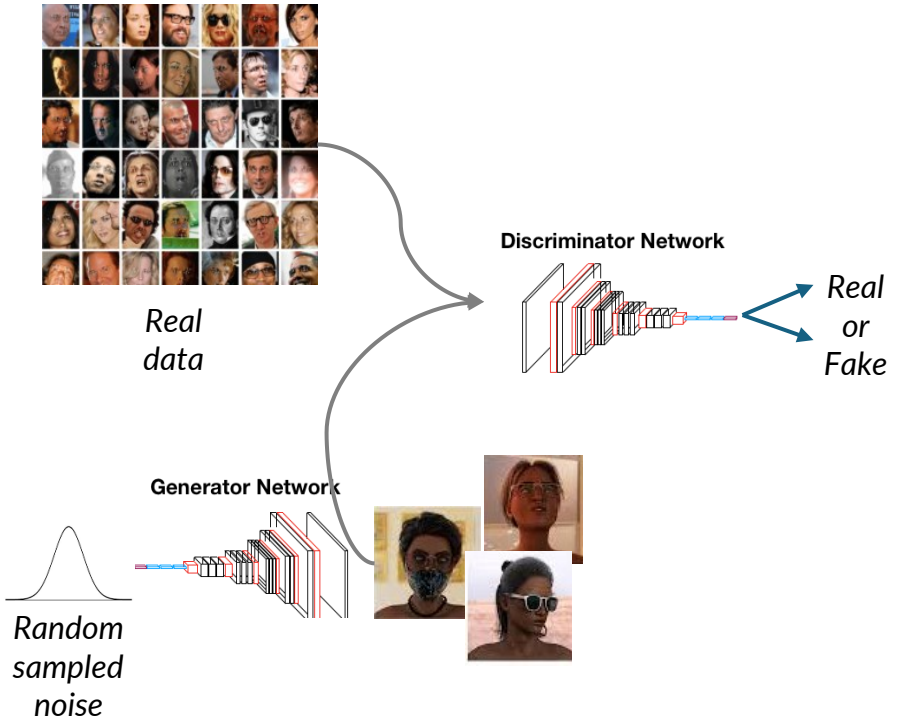



# PREVENTING FAILURES

- **Feature Matching**
  - Statistics of generated images should match statistics of real images
  - Discriminator produces multidimensional output, a “statistic” of data
  - Generator trained to minimise L2 between real and generated data, discriminator trained to maximise L2 between real and generated data
- **Historical averaging**
  - Dampen oscillations by encouraging updates to converge to a mean GANs frequently create a cycle or experience oscillations
  - Add a term to reduce oscillations that encourage the current parameters to be near a moving average of the parameters
- **Minibatch Discrimination**
  - Discriminator evaluates multiple samples jointly
  - Enables detection of lack of diversity across generated samples
  - Append sample-to-sample similarity features to discriminator layers
  - Reduces mode collapse

# LIMITATIONS

- GAN training is highly sensitive to architecture choices, learning rates, and data balance
- It doesn't guarantee full data coverage or stable convergence
- It only learns to map noise to outputs that fool the discriminator, without understanding
- Many tasks require transforming one image into another while preserving content or identity:
  - Increasing resolution, converting grayscale to colour, changing attributes of an image
- However, standard GAN enables new classes of models:
  - Conditional GANs
  - Style-based GANs
  - Image-to-image translation GANs

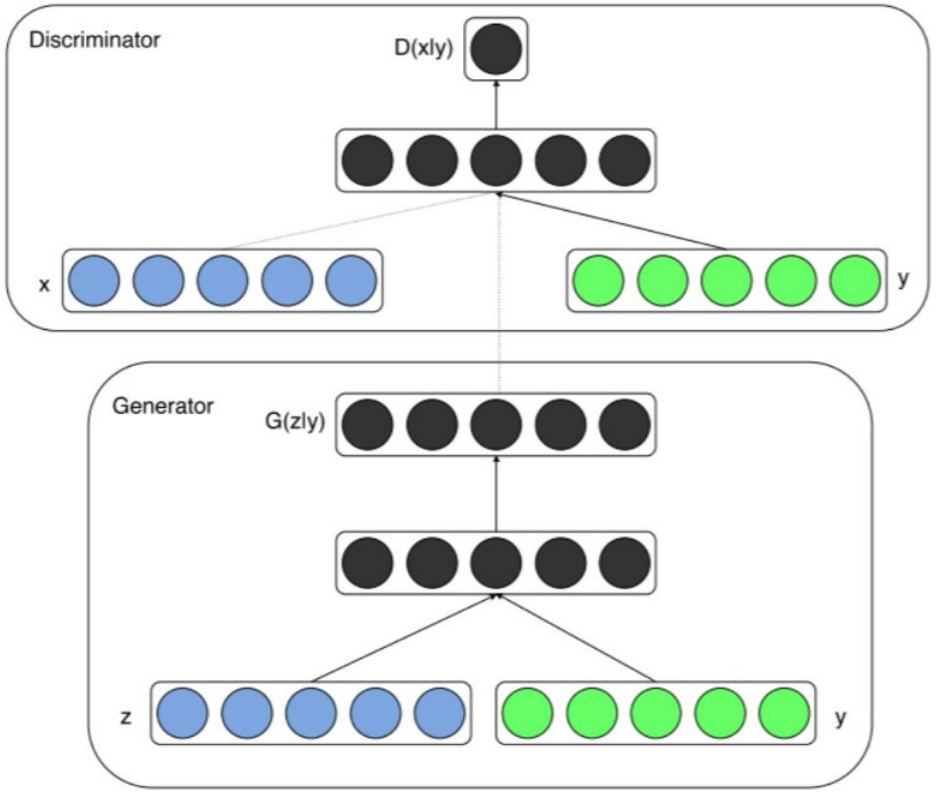



The generator samples random noise and is never told

- what it is generating
- which attributes should be preserved
- what structure should remain unchanged

# CONTROLLING GENERATED SAMPLES

- Discriminative models learn the conditional distribution $P(Y \mid X)$
- Generative models learn how data is generated and can sample new data
- Generative models may model:
  - $P(X)$ (unconditional generation)
  - $P(X \mid Y)$ (conditional generation)
- Vanilla GANs implicitly model the data distribution $P(X)$ and generate new samples after training
- A **conditional GAN** models $P(X|Y)$ and enables controlled generation
- It is implemented by providing the labels (condition) $Y$ as input to the generator and discriminator



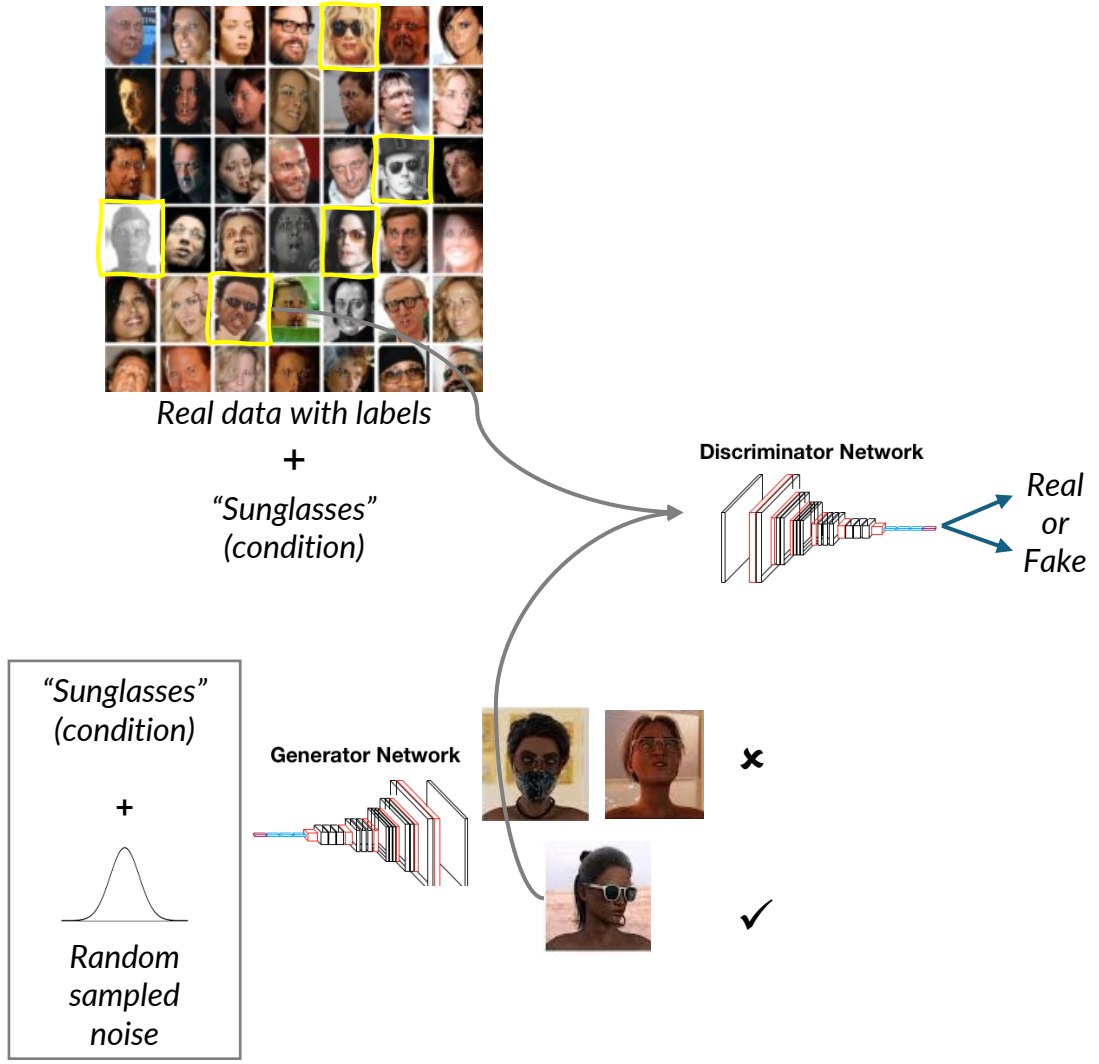

Conditional GAN

# CONDITIONAL GAN

- Conditional GANs modify the GAN setup by conditioning generation on an input
- The generator now learns:

$$(z, y) \rightarrow x$$

  - where $c$ is a condition (class label, image, or attribute)
- The discriminator also evaluates realism given the same condition
- Generation is no longer arbitrary
- Outputs are tied to
  - a label
  - an image
  - a specified condition
- However, conditional GANs typically require paired data



# PAIRED DATA

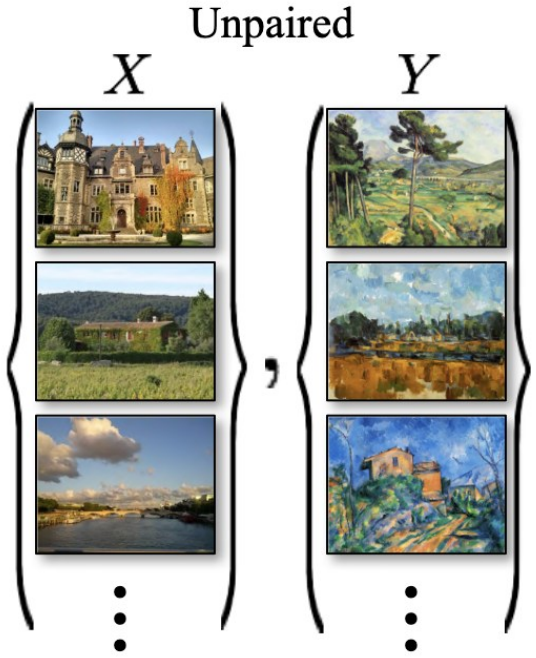
- Paired data consists of corresponding input–output examples
- Each input sample has a ground-truth target in another domain
- Pairs represent the same underlying content under different representations
- cGANs learn a direct mapping from a condition to an output
- Training assumes paired examples:
  - the same scene in two domains
  - the same object under two representations
- The discriminator evaluates realism relative to the given input
- Training relies on explicit supervision: generated output is compared against the paired target
- Without pairs, the model has no signal for which output corresponds to which input




- Sketch → photo
- Edge map → image
- Low-resolution → high-resolution

# UNPAIRED DATA

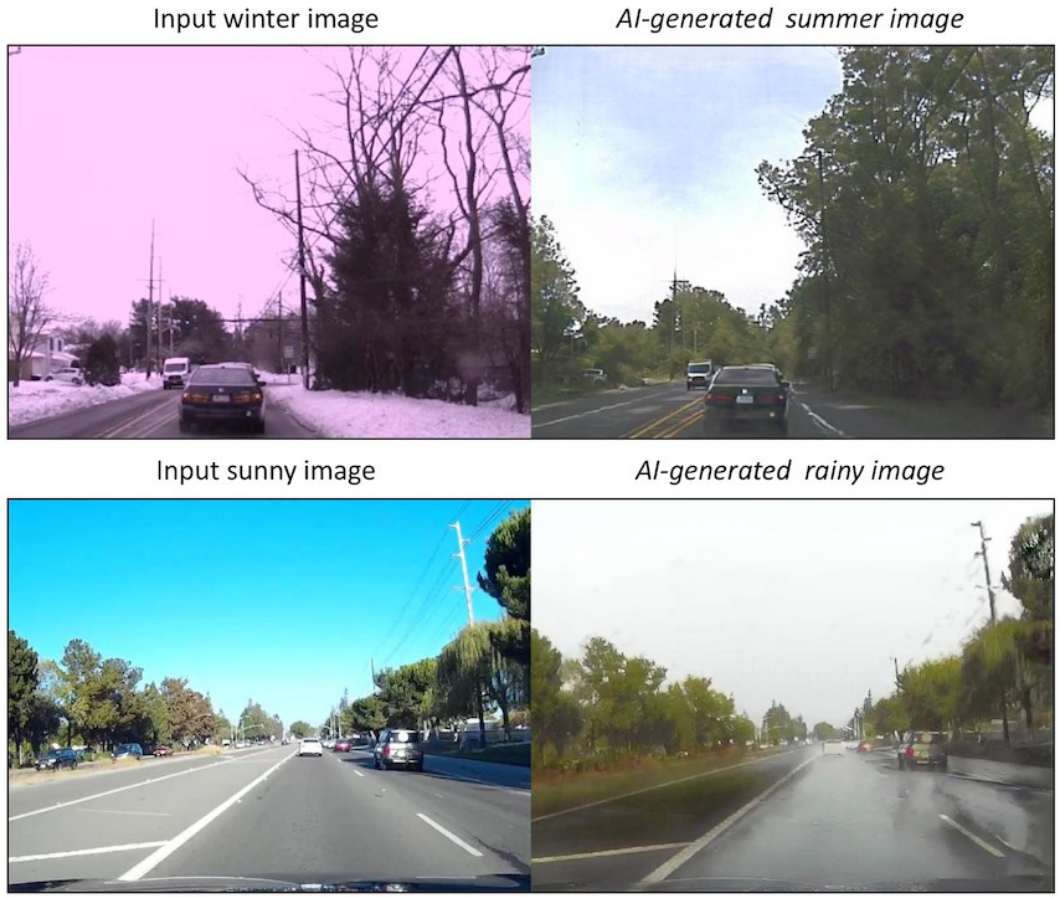
- Unpaired data consists of independent sets from each domain
- No one-to-one correspondence between samples
- Supervision available only at the distribution level
- Unpaired data is found in abundance, whereas paired data
  - is expensive to collect
  - is rare in real applications
  - is sometimes impossible to obtain
  - requires precise alignment between domains
- So, how can we learn conditional image translation when paired examples are not available?
- CycleGAN extends conditional GANs to the unpaired setting
- Conditional GANs learn controlled realism, CycleGAN learns controlled realism without pairs




- Photos ↔ paintings
- Summer ↔ winter scenes
- Horses ↔ zebras

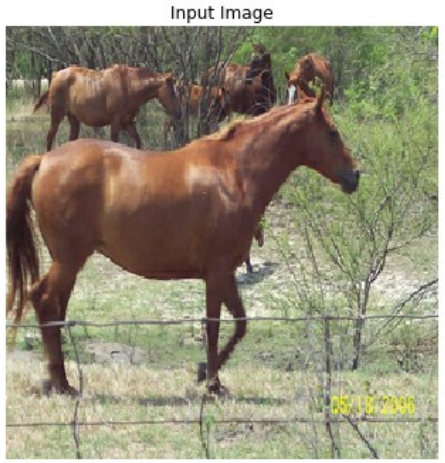
# IMAGE TRANSLATION

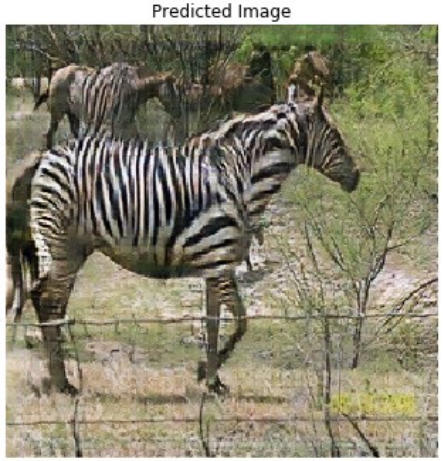
- Image translation is the task of mapping an image from one domain to another
- The goal is to
  - change visual appearance or style
  - preserve underlying structure and semantic content
- This is structure-preserving transformation, and not generation from random noise sampling
- As we saw, image to image translation is either **paired** or **unpaired**
- For example,
  - Photo → painting
  - Summer → winter
  - Horse → zebra
  - Grayscale → colour



# WHY cGANs ARE NOT ENOUGH

- Conditional GANs require paired data, with inputs having a target image
- In many real-world problems, paired samples do not exist, as it may be expensive or even impossible
  - For example, if you want to translate horse images to look like zebras
  - There is no paired dataset of zebras and horses
  - So, no easy discriminative method to train zebras from horses
- We often only have
  - a set of images from domain A
  - a set of images from domain B
  - no one-to-one correspondence
- We want to perform image translation by preserving content and structure, but we do not have paired examples
- Using CycleGANs can train the distributions to match

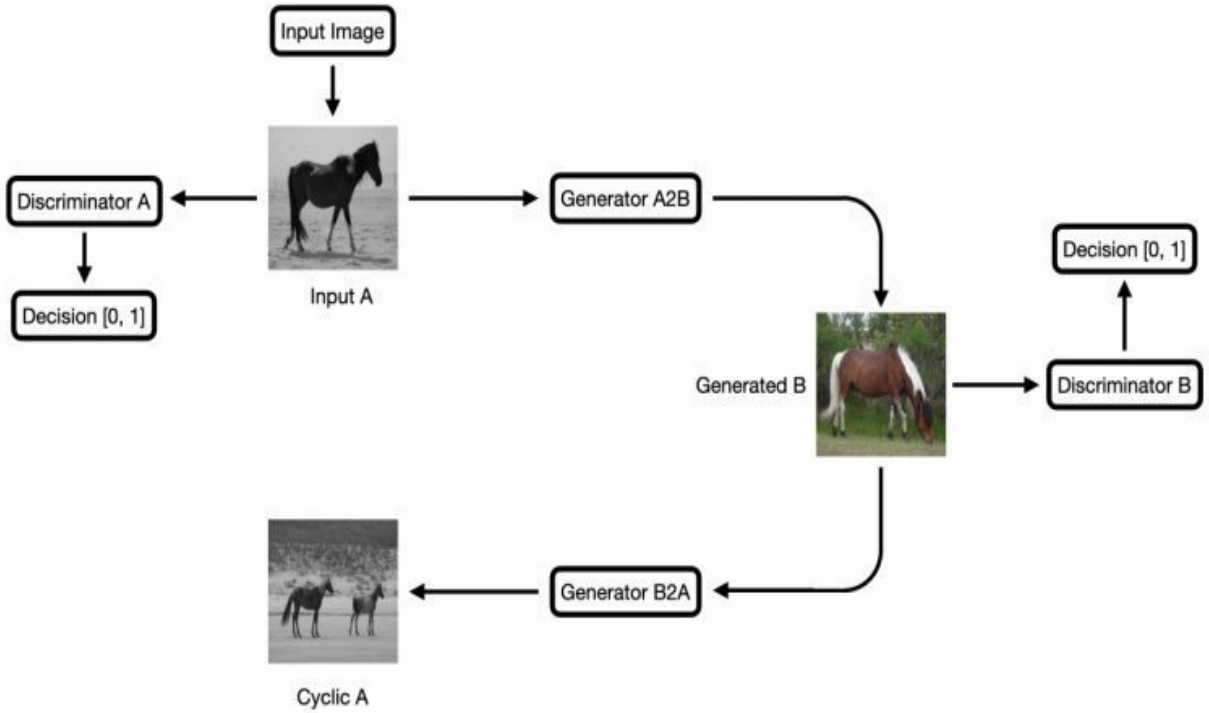


# CYCLE GAN

CycleGAN (**Cycle-Consistent Generative Adversarial Network**) is a type of GAN that learns to translate images from a source domain *A* to a target domain *B* (and vice-versa) without requiring paired examples

- A CycleGAN consists of two generators and two discriminators, creating a "cycle" of translation
- Two Generators
  - ***GeneratorA2B***: This generator network translates images from domain *A* to domain *B*
  - ***GeneratorB2A***: This generator network translates images from domain *B* to domain *A*
- Two Discriminators
  - **Discriminator *G***: This discriminator network distinguishes between real images from domain *A* and fake images generated by *GeneratorA2B*
  - **Discriminator *F***: This discriminator network distinguishes between real images from domain *B* and fake images generated by *GeneratorB2A*



# USE CASES OF CYCLE GAN

*A standard example of unpaired image-to-image translation is making this horse video into a zebra video (frame-by-frame)*

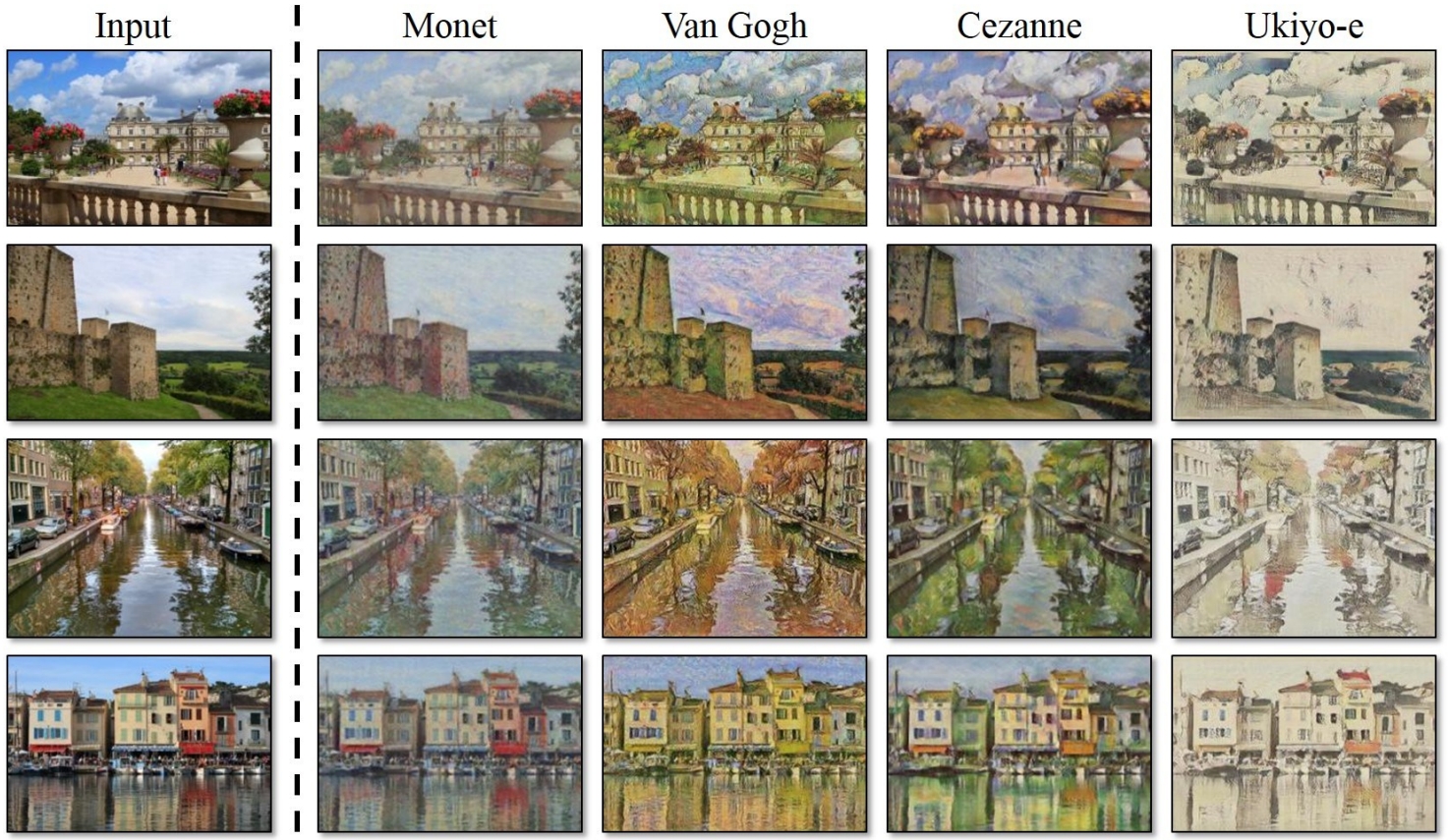
*This is done using CycleGAN with unpaired datasets of horse and zebra images*



# USE CASES OF CYCLE GAN

*Another application is translating real photographs to paintings of different styles – also known as **style transfer***

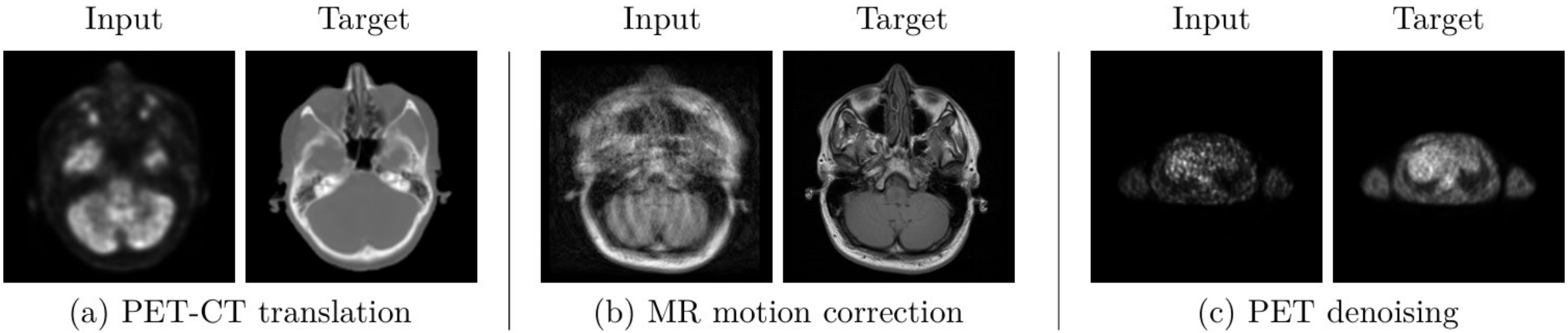
*Style transfer refers to techniques that generate an image which preserves the semantic content and spatial structure of one image while adopting the visual appearance (texture, colour, and lighting) of another*



# USE CASES OF CYCLE GAN

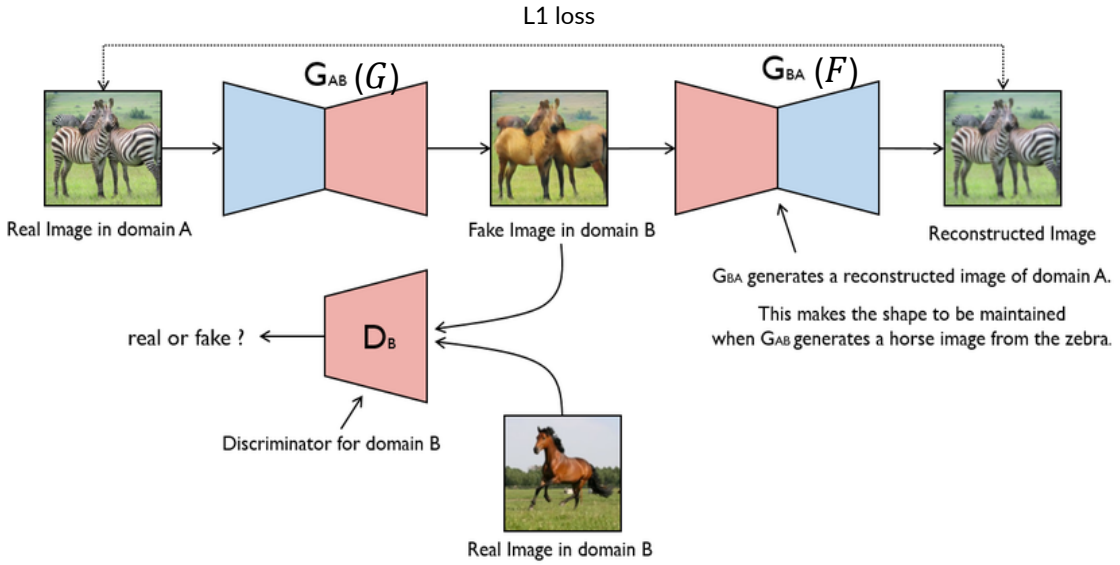
*MedGAN is a GAN-based framework, inspired by cycle-based translation models like CycleGAN*

*It is used for medical image translation and reconstruction, and can be applied to tasks such as segmentation and denoising*



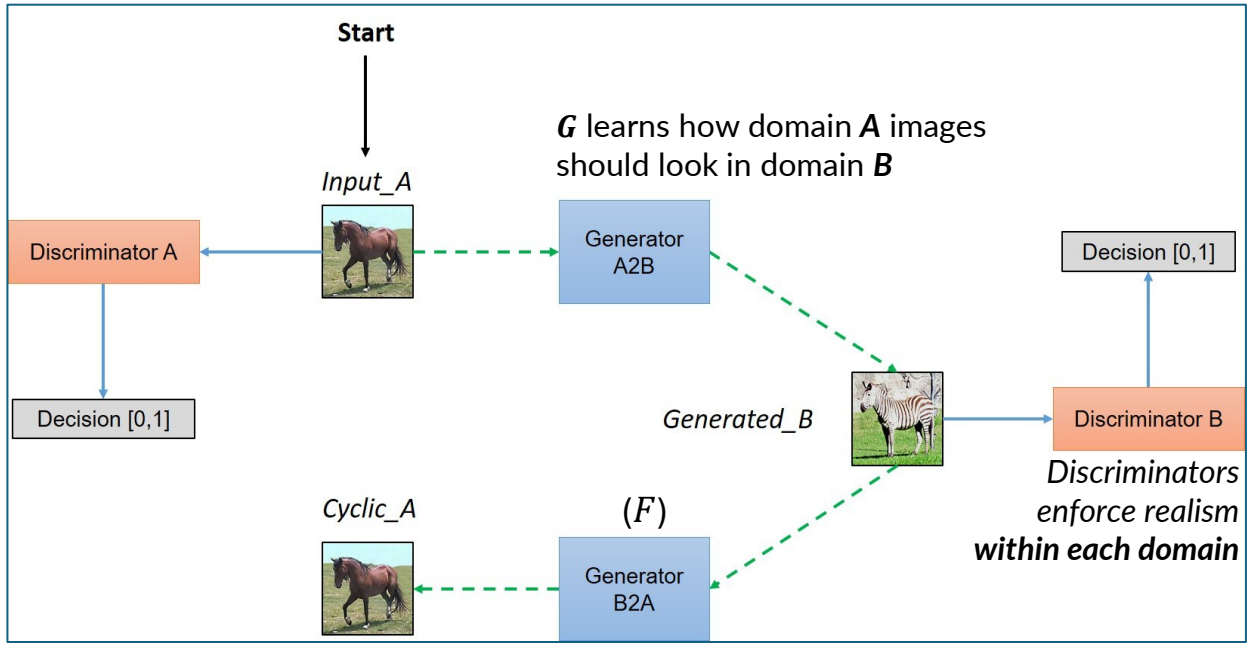
# CYCLEGAN WORKING AND ARCHITECTURE

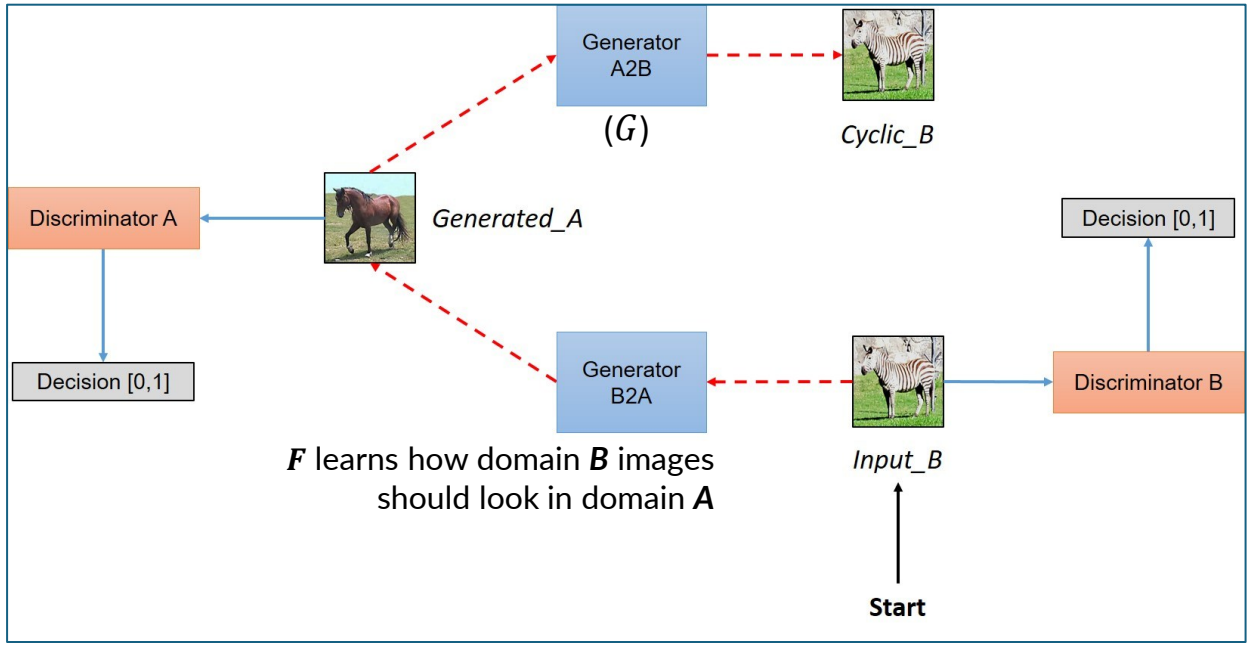
- CycleGAN extends GANs to unpaired image-to-image translation
- It learns mappings between two domains, using two generators:
  - $G: A \rightarrow B$
  - $F: B \rightarrow A$
- No paired examples are used, and learning is driven by consistency constraints
- Also, two discriminators are used:
  - $D_{\mathrm{B}}$ for real vs fake images in domain B
  - $D_{\mathrm{A}}$ for real vs fake images in domain A
- Each generator is trained with its corresponding discriminator, forming two coupled GANs
- Generators are usually fully convolutional (e.g. ResNet or U-Net)
- Architectures are like vanilla GANs; difference is the dual generator–discriminator setup



# CYCLES

- CycleGAN actually forms two translation cycles:
  - **Forward cycle:** real image from domain *A* → domain *B* → back to domain *A*
  - **Backward cycle:** real image from domain *B* → domain *A* → back to domain *B*
- A real image from domain ***A*** is translated to ***B*** and then mapped back to ***A***
- A real image from domain ***B*** is translated to ***A*** and then mapped back to ***B***

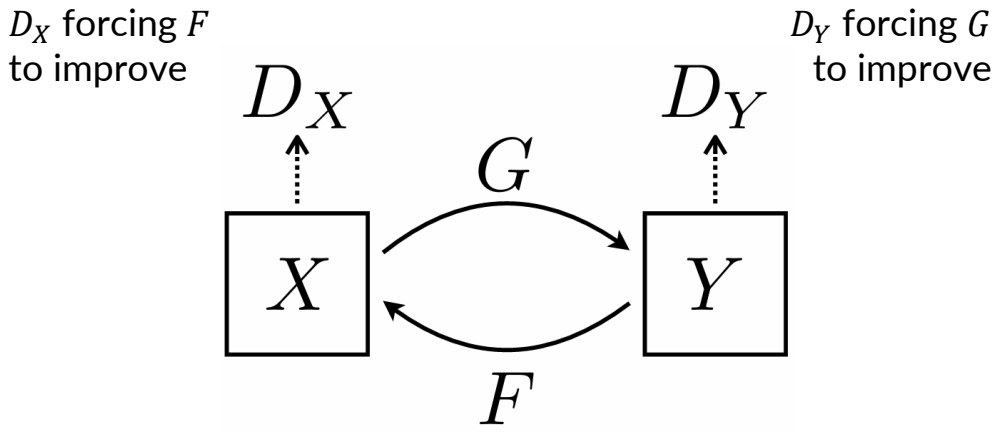



*Simplified view of CycleGAN architecture*

# CYCLE CONSISTENCY

- So, the overall CycleGAN process is two GANs coupled together by an additional constraint
  - $A \rightarrow G \rightarrow B \rightarrow F \rightarrow A$
  - $B \rightarrow F \rightarrow A \rightarrow G \rightarrow B$
  - Discriminators on both domains
- Cycle consistency constrains the model such that forward–backward translation reconstructs the original image
- The cycle consistency loss penalises discrepancies between original images and their cyclic reconstructions




*Setting of two mapping functions and associated generators*

# THREEFOLD LOSS FUNCTION

- CycleGAN learns without labels and optimises three components, which work as self-supervised learning
  - **Adversarial Loss** (standard GAN loss)
  - **Cycle-Consistency Loss** (the core of CycleGAN)
  - **Identity Loss** (optional, when input and target domains are similar)

- Adversarial loss in CycleGAN ensures **local realism** through discriminators
- They ensure outputs lie on the target domain manifold
- The goal is to ensure translated images:
  - look realistic in the target domain
  - match the target domain distribution
- Adversarial loss alone does not preserve content

- Adversarial loss is used twice, once for domain ***B*** and once for ***A***
- $D$ has loss for both real and fake data, whereas $G$ and $F$ get loss only for fake data
- For generator $G: A \rightarrow B$ and discriminator $D_{\mathrm{B}}$

$$\mathcal{L}_{\mathrm{GAN}}(G, D_{\mathrm{B}}, A, B) = \mathbb{E}_{y \sim p_{\mathrm{data}}(\mathrm{B})}[\log D_{\mathrm{B}}(y)] + \mathbb{E}_{x \sim p_{\mathrm{data}}(\mathrm{A})}\left[\log\left(1 - D_{\mathrm{B}}(G(x))\right)\right]$$

- Similarly, for $F: B \rightarrow A$ and discriminator $D_{\mathrm{A}}$,

$$\mathcal{L}_{\mathrm{GAN}}(G, D_A, B, A)$$

# CYCLE CONSISTENCY LOSS

- Adversarial loss enforces realism, but allows arbitrary mappings
- Cycle consistency enforces structure and semantic preservation
- Translating an image forward and backward should recover the original
- It enforces **global structure preservation** and prevents generators from making irreversible or destructive changes
- Replaces paired supervision with a self-consistency constraint
- For domain ***A***

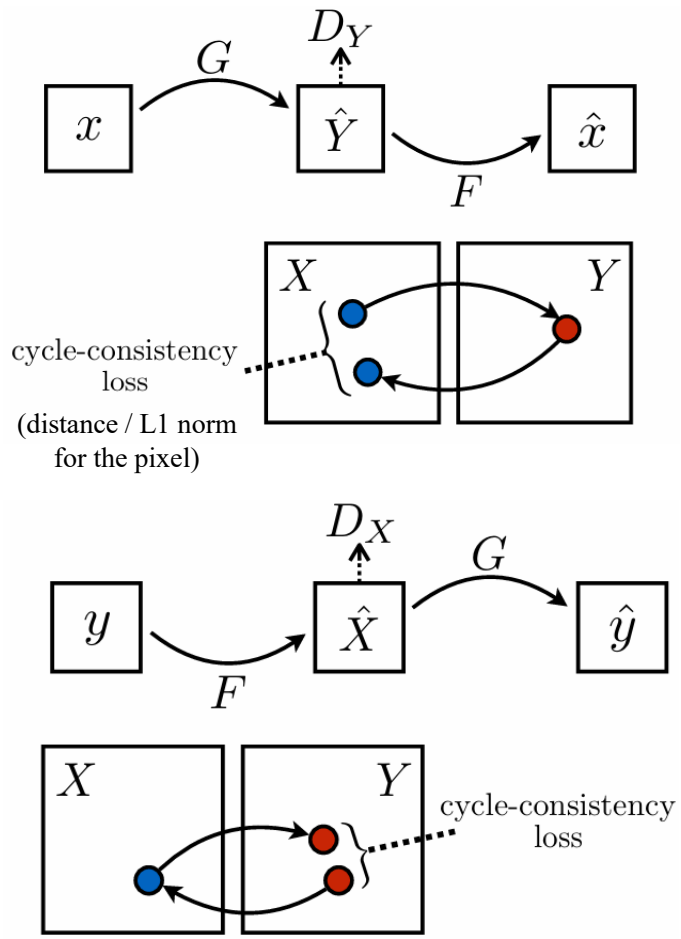
$$\mathcal{L}_{\text{cyc(A)}} = \mathbb{E}_{x \sim p_{\text{data}}(\text{A})}[\|F(G(x)) - x\|_1]$$

- For domain ***B***

$$\mathcal{L}_{\text{cyc(B)}} = \mathbb{E}_{y \sim p_{\text{data}}(\text{B})}[\|F(G(y)) - y\|_1]$$

- Total cycle consistency loss

$$\mathcal{L}_{\text{cyc}} = \mathcal{L}_{\text{cyc(A)}} + \mathcal{L}_{\text{cyc(B)}}$$

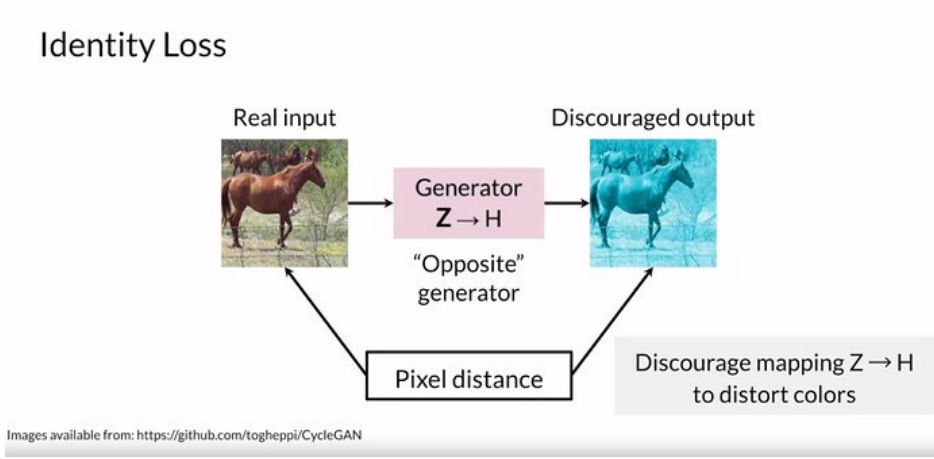


# IDENTITY LOSS

- Cycle consistency alone may still alter images unnecessarily
- Identity loss discourages changes when the input is already in the target domain
- It replaces paired supervision with a self-consistency constraint
- It encourages minimal transformation and preserves colour composition, intensity statistics etc.
- It acts as a local regulariser that complements global cycle consistency
- We can write identity loss as,

$$\mathcal{L}_{\text{id}}(G, F) = \mathbb{E}_{y \sim p_{\text{data}}(\text{B})}[\|G(y) - y\|_1] + \mathbb{E}_{x \sim p_{\text{data}}(\text{A})}[\|F(x) - x\|_1]$$

*$\mathcal{L}_{\text{cyc}}$ uses a full cycle (e.g., zebra to horse to zebra) while $\mathcal{L}_{\text{id}}$ uses only one-direction (e.g., horse to horse)*




We can now write the complete loss function for CycleGAN as

$$\mathcal{L}_{\text{CycleGAN}} = \mathcal{L}_{\text{GAN}} + \lambda_{\text{cyc}}\mathcal{L}_{\text{cyc}} + \lambda_{\text{id}}\mathcal{L}_{\text{id}}$$

Here, $\lambda$ is the weighting factor for $\mathcal{L}_{\text{cyc}}$ and $\mathcal{L}_{\text{id}}$ losses

# LIMITATIONS AND ASSUMPTIONS

- CycleGAN assumes source and target domains share a common underlying structure
- Translation mainly modifies texture, colour, or style, not geometry
- Cycle consistency enforces reversibility, but not semantic correctness
- CycleGAN fails when domains differ significantly, leading to semantic drift
- Objects may appear, disappear, or deform unnaturally
- Learns correlations rather than meaning, dataset biases are learned and amplified
- Translation quality depends heavily on dataset quality







These limitations motivate generative models that offer better stability, coverage, and controllability, leading to diffusion-based approaches